UNIFAAT 2026.2

10 - Estrutura de Dados

Prof. Luan Lourenço

Dado, tipo e binário

```
# dado: valor bruto, sem contexto
1234  ·  "Ana"  ·  8.5
RA? preço? ano? sozinho não diz nada

# tipo: o que o valor é e o que se faz com ele
2 + 3 dá 5
"2" + "3" dá "23"
```

src/10/binario_e_estrutura.py

```
bin(65)   0b1000001

for letra in "Ola":
    print(letra, ord(letra), bin(ord(letra)))
O 79 0b1001111
l 108 0b1101100
a 97 0b1100001
```

- **Inteiro** (Aula 03): número na base 2. **String** (Aulas 03 e 07): cada caractere é um número da tabela, e o número vira binário
- O `65` e o `"A"` são **os mesmos bits**: o tipo é a lente que lê

O computador não tem "número", "texto" nem "lista": só **sequências de bits**. A **estrutura de dados** é como esses valores se organizam.

Estrutura: como os binários se organizam

```
# memória: bytes numerados (endereços)
100  101  102  103
 79   108    97    ...

# lado a lado: a posição vira conta
endereço = início + posição × tamanho
```

src/10/binario_e_estrutura.py

```
dados = bytes([79, 108, 97])
print(dados)   b'Ola'
print(dados[0], dados[2])   79 97
```

Acessar `notas[2]` é rápido em qualquer tamanho. Cada estrutura é **boa para uma coisa e ruim para outra**: escolher bem é metade do programa.

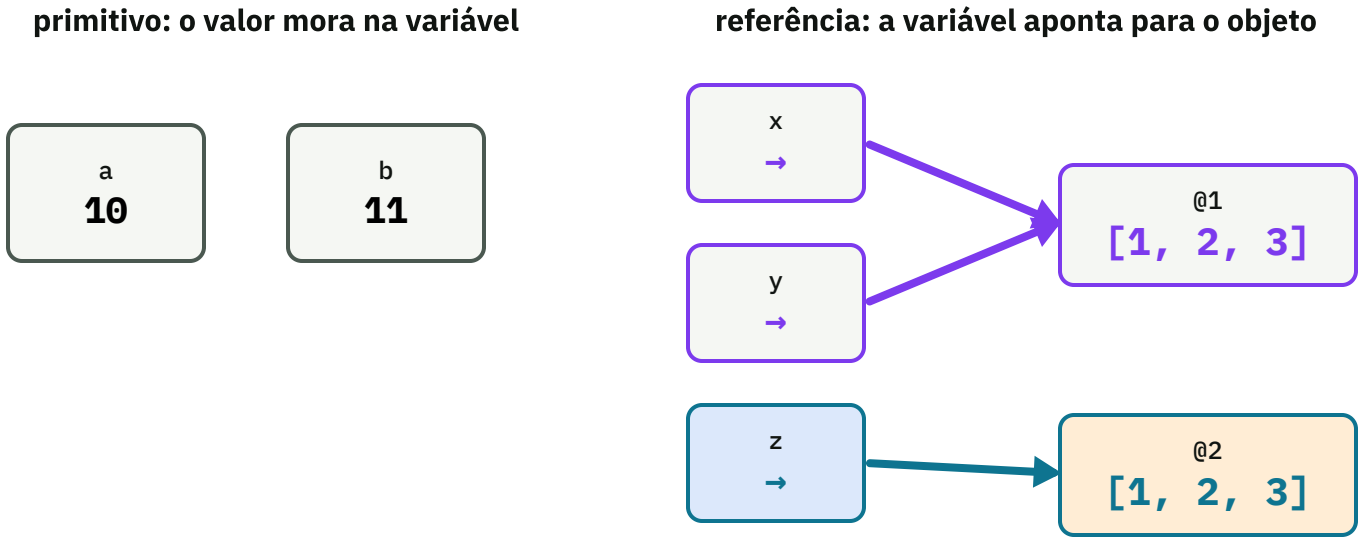
Tipo primitivo x tipo de referência

	primitivo	referência
a variável guarda	o **valor** em si	o **endereço** do objeto
`b = a` **copia...**	o valor: dois independentes	o endereço: dois nomes, **um objeto**
exemplos	`int`, `float`, `bool`, `str`	`list`, `dict`, `tuple`, objetos de classe
tamanho	pequeno e fixo	varia: pode crescer

Em Python **tudo é objeto**: não há primitivo "puro" como o `int` do Java. Os que **se comportam** como primitivo são os **imutáveis**. Os de referência são as **estruturas de dados**.

O `b = a` que engana

EXEMPLO



`z = x.copy()`
copiar de verdade cria outro objeto, com outro endereço

`is` pergunta "é o **mesmo objeto**?"; `==` pergunta "têm o **mesmo valor**?". É o mesmo mecanismo do `self` e do `pai` da árvore de arquivos.

O que é ser nativo

Já vem na linguagem: sem `import`, sem biblioteca, sem classe sua.

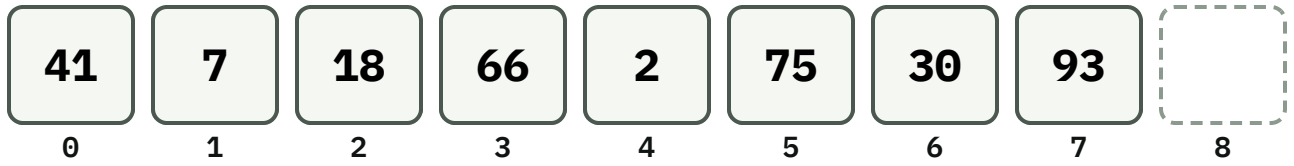
linguagem	o que traz pronto
C	array de tamanho fixo e `struct`; sem dicionário
Java	`int[]` na linguagem; `ArrayList` e `HashMap` vêm do `java.util`; sem tupla
JavaScript	`Array` e `Object` (e `Map`); sem tupla
PHP	o `array` é lista e dicionário ao mesmo tempo
Python	**`list`, `tuple` e `dict` direto na sintaxe**

A ideia é a mesma em todas; muda o nome e o quanto vem de graça.

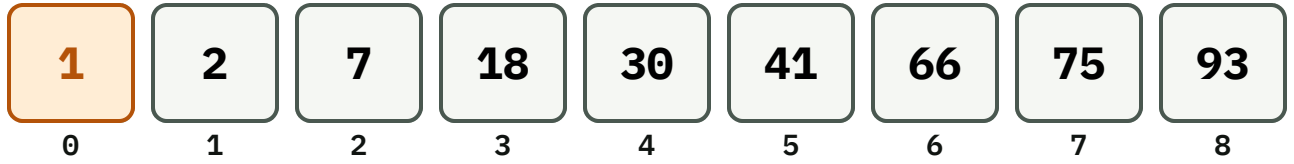
Leitura x escrita: ganhar de um lado é perder do outro

ESCRITA

passos: 9



lista solta: sem ordem



lista ordenada

ordenada.insert(0, 1)
perde: 9 passos só para escrever (a ordem tem preço)

LEITURA x ESCRITA

ESCREVER **solta.append(x)** O(1)
solta: escrever é largar no fim

LER **x in solta** O(n)
solta: ler é olhar um por um

LER **busca binária** O(log n)
ordenada: ler descarta metade

ESCREVER **ordenada.insert()** O(n)
ordenada: escrever abre espaço

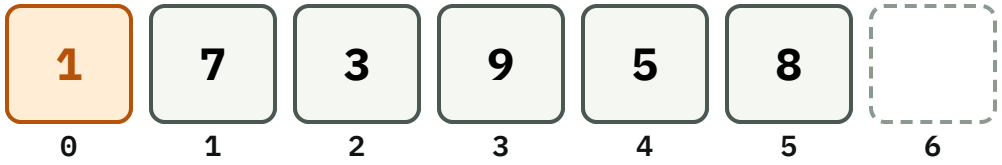
O(1): 1 passo, não importa o tamanho. **O(n)**: os passos crescem com a quantidade de dados.

Leitura consulta sem mudar; **escrita** inclui, altera ou remove. As duas são **antagônicas**: organizar para ler rápido (ordem, índice, hash) dá trabalho na escrita; facilitar a escrita obriga a varrer na leitura.

Lista: o array do Python (vetor = array)

ESCRITA

passos: 6



notas.insert(0, 1)
só agora o 1 cabe: 6 passos para escrever no início

LEITURA x ESCRITA

LER **notas[i]** O(1)
por posição: vai direto

LER **x in notas** O(n)
por valor: olha um por um

ESCREVER **append(x)** O(1)
no fim: cabe no espaço livre

ESCREVER **insert(0, x)** O(n)
no início ou no meio: empurra todo mundo

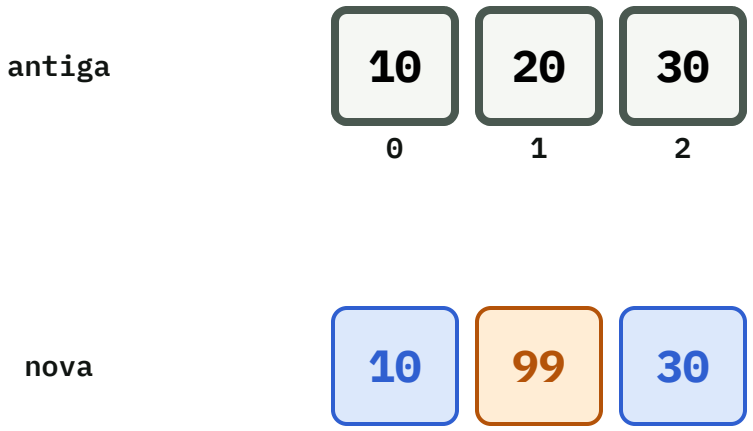
O(1): 1 passo, não importa o tamanho. **O(n)**: os passos crescem com a quantidade de dados.

Ganha: ler por posição. **Perde:** escrever no início ou no meio.

Tupla: imutável

ESCRITA

passos: 3



(t[0], 99, t[2])
a tupla antiga continua intacta: alterar custa copiar tudo

LEITURA x ESCRITA

LER **t[i]** — O(1)
por posição: igual à lista

ESCREVER **t[i] = x** — proibido
escrever não existe: TypeError

ESCREVER **nova tupla** — O(n)
"alterar" = copiar tudo para outra

EXEMPLO **chave de dict** — ganho
imutável: o hash nunca muda

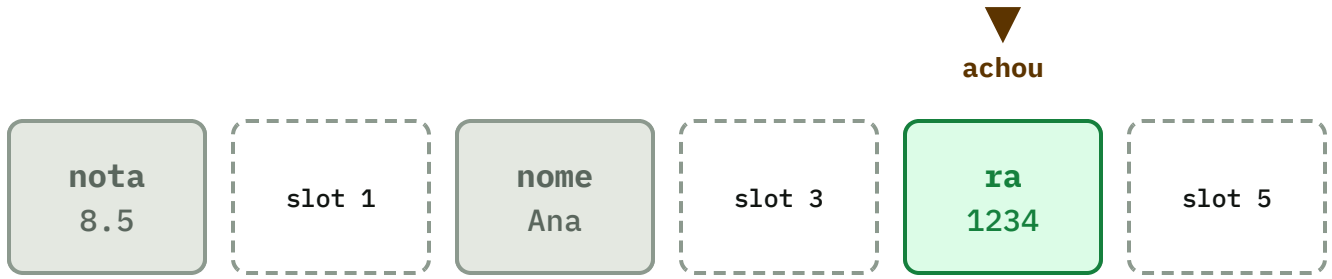
O(1): 1 passo, não importa o tamanho. **O(n)**: os passos crescem com a quantidade de dados.

Ganha: segurança, e vira chave de dicionário. **Perde:** a escrita: "alterar" é copiar tudo.

Dicionário: o famoso HashMap

LEITURA

passos: 5



1234 in aluno.values()
achou, mas passou por 5 slots: buscar pelo valor não usa o hash

LEITURA x ESCRITA

LER **d[chave]** O(1)
por chave: o hash vai direto ao slot

LER **x in d.values()** O(n)
por valor: olha slot por slot

ESCREVER **d[chave] = x** O(1)
o hash diz onde guardar

PREÇO **memória** preço
slots vazios reservados

O(1): 1 passo, não importa o tamanho. **O(n)**: os passos crescem com a quantidade de dados.

Ganha: ler e escrever pela chave em 1 passo. **Perde:** memória extra e a busca pelo valor.

Relacionamento: um objeto aponta para outro

src/10/relacionamentos.py

```
class Turma:
    def __init__(self, nome, professor):
        self.professor = professor   aponta, não copia

luan = Professor("Luan")
ads = Turma("ADS", luan)
es = Turma("ES", luan)
ads.professor is es.professor   True
```

```
# "é um": herança (Aula 09)
Arquivo é um Item

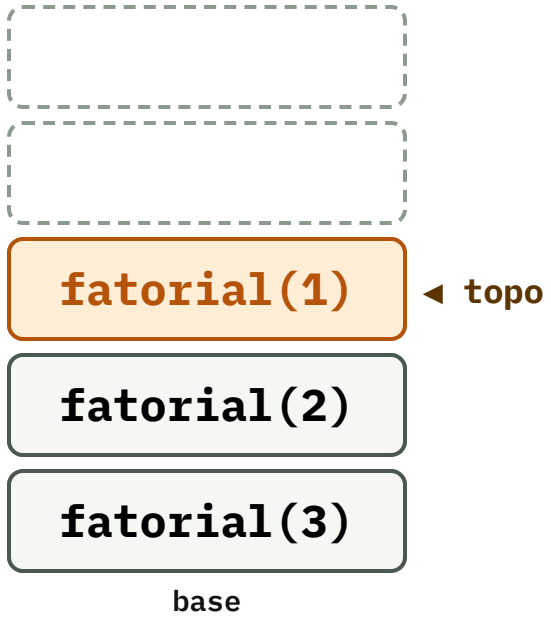
# "tem um": composição / associação
Turma tem um Professor
Pasta tem filhos (um para muitos)
```

Um só professor na memória, duas turmas guardando o **endereço** dele. **Uma lista de objetos já é uma estrutura de dados.**

Pilha: o último a entrar é o primeiro a sair

EXEMPLO

passos: 3



fatorial(3)
fatorial(2) chama fatorial(1): a última a entrar...

LEITURA x ESCRITA

ESCREVER **empilhar(x)** O(1)
só pelo topo

LER **topo()** O(1)
só o de cima

LER **ler o fundo** O(n)
tira todos os de cima antes

EXEMPLO **pilha de chamadas** LIFO
o último a entrar sai primeiro

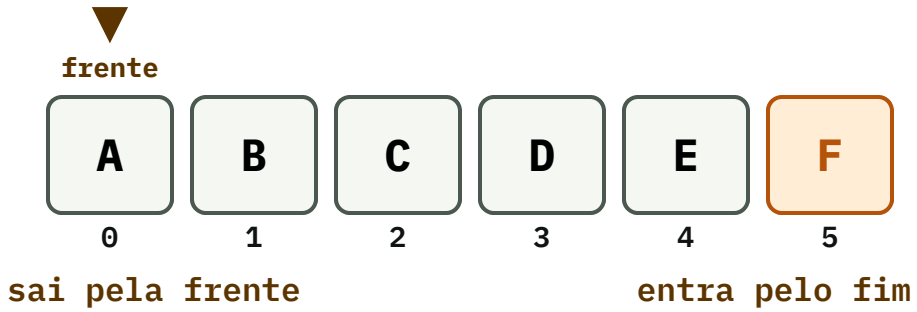
O(1): 1 passo, não importa o tamanho. **O(n)**: os passos crescem com a quantidade de dados.

Ganha: simplicidade, o topo em 1 passo. **Perde:** o acesso ao meio e ao fundo.

Fila: o primeiro a entrar é o primeiro a sair

ESCRITA

passos: **1**



fila.enfileirar("F")
entra pelo fim, sem mexer em ninguém: 1 passo

LEITURA x ESCRITA

ESCREVER **enfileirar(x)** — O(1)
entra pelo fim

LER **desenfileirar()** — O(1)
sai pela frente (deque)

ESCREVER **lista.pop(0)** — O(n)
uma lista comum desloca todos

LER **fila[i]** — O(n)
o meio: caminha desde a ponta

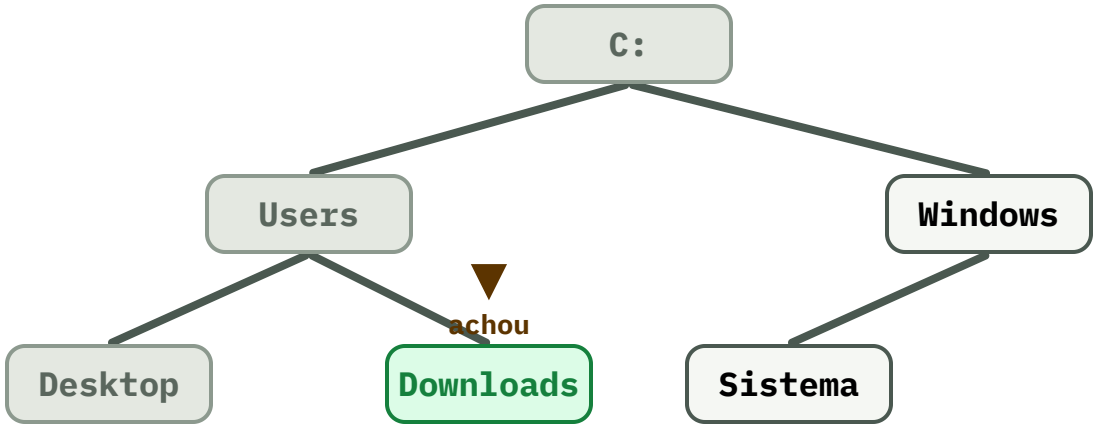
O(1): 1 passo, não importa o tamanho. **O(n)**: os passos crescem com a quantidade de dados.

Ganha: a ordem de chegada, frente e fim em 1 passo. **Perde:** o meio (e o `pop(0)` da lista comum).

Árvore: hierarquia, um pai por nó

LEITURA

passos: 4



achar "Downloads"
achou, depois de passar por 4 pastas: sem o caminho, varre tudo

LEITURA x ESCRITA

LER	por caminho	O(prof.)
	um passo por nível	
ESCREVER	adicionar filho	O(1)
	com o pai na mão	
ESCREVER	mover_para()	O(1)
	troca 1 ponteiro, seja qual for o tamanho	
LER	por nome	O(n)
	sem o caminho, varre tudo	

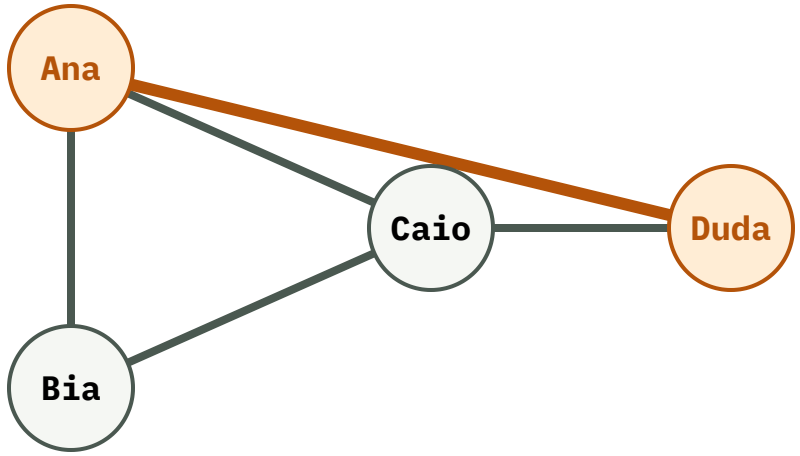
O(1): 1 passo, não importa o tamanho. **O(n)**: os passos crescem com a quantidade de dados.

Ganha: hierarquia e escrita local barata (mover um galho é 1 ponteiro). **Perde:** a busca global, sem o caminho.

Grafo: nós e ligações, sem hierarquia

ESCRITA

passos: 2



dicionário de vizinhos

Ana: [Bia Caio Duda]

Bia: [Ana Caio]

Caio: [Ana Bia Duda]

Duda: [Caio Ana]

g.ligar("Ana", "Duda")
...depois na da Duda (2 passos): a ligação vale nos dois sentidos

LEITURA x ESCRITA

ESCREVER	**ligar(a, b)** — um item em cada lista	O(1)
LER	**vizinhos(a)** — a lista já está pronta	O(1)
LER	**ligados(a, b)** — varre a lista de vizinhos	O(grau)
ESCREVER	**remover(a, b)** — acha nas duas listas	O(grau)
ESCREVER	**git commit** — o Git é um grafo: nó + aresta pro pai	O(1)
LER	**git log** — segue os pais, commit a commit	O(n)

O(1): 1 passo, não importa o tamanho. **O(n)**: os passos crescem com a quantidade de dados.

Ganha: modela qualquer relação e lista vizinhos direto. **Perde:** "A liga a B?" e remover varrem a lista. **Caso real: o Git.** Commit é nó, "pai" é aresta, merge é nó com dois pais. A árvore é um grafo sem ciclos.

Leitura x escrita: o resumo

estrutura	lê rápido	escreve rápido	o preço
lista	por posição	no fim	início e meio empurram todos
tupla	por posição	não escreve	alterar = copiar tudo
dicionário	por chave	por chave	memória; valor varre tudo
pilha	o topo	o topo	fundo e meio custam n
fila	a frente	o fim	o meio custa n
árvore	por caminho	filho, mover galho	busca por nome varre tudo
grafo	vizinhos	ligar	"A liga B?" e remover varrem

Não existe estrutura que ganhe em tudo. A pergunta é sempre: **vou mais ler ou mais escrever, e o quê?**

tarefa

TF 10: o MiniGit

Num repositório novo `tf-10`, com venv e `.gitignore`, criar a classe `MiniGit` em `minigit.py`:

- → **Estruturas:** dicionários (pais, mensagens, branches), lista, tupla e uma **fila** (`deque`) no `log()`
- → **Escrita:** `commitar`, `criar_branch`, `trocar_branch`, `mesclar` (commit com **dois pais**)
- → **Leitura:** `mensagem`, `pais`, `log`, `buscar`, `merges`
- → **Contar passos:** a property `passos` diz o custo da última operação; medir `commitar` (sempre 2) e `log` (cresce) com 10, 100 e 1000 commits
- → 4 tentativas que **falham**, README respondendo leitura x escrita e 5+ commits. Sem `else`, atributos privados

Entrega: link do repositório público no GitHub, pelo formulário abaixo —
`docs.google.com/forms/d/e/1FAIpQLSfW4bh9zbk6fae3Z1XzzujO2wRlfEq4il7axAL9jx2Se-BupQ/viewform`

Prazo: **04/11, até 23:59**.